

Software Architecture and Design Specification

Project 27 | Project Management System | Version 1.0 | 28 September 2026

Prepared for the Software Engineering mini-project. Team and approval details may be completed by the submitting group.

1 Introduction

This specification maps the SRS for Project 27 to components, data, interfaces, security controls and interaction flows. It provides implementable decisions and test hooks for the accompanying test plan.

2 Document overview

Read section 3 for components and security boundaries, then section 4 for task flows and API contracts. Related baselines: Project27_SRS.docx and Project27_Test_Plan.docx. FR, NFR and SR IDs refer to SRS version 1.0.

3 Architecture

3.1 Goals and constraints

Keep authorization centralized, preserve audit history, calculate progress from stored task data, and support the defined 500-task load. A Django application with a relational database is proposed within the project-list stack allowance.

3.2 Stakeholders and concerns

Members need clear assignments and workflow; Owners need access control and status visibility; developers need maintainable modules; testers need deterministic fixtures and traceable endpoints.

3.3 UML component diagram

Layered component view

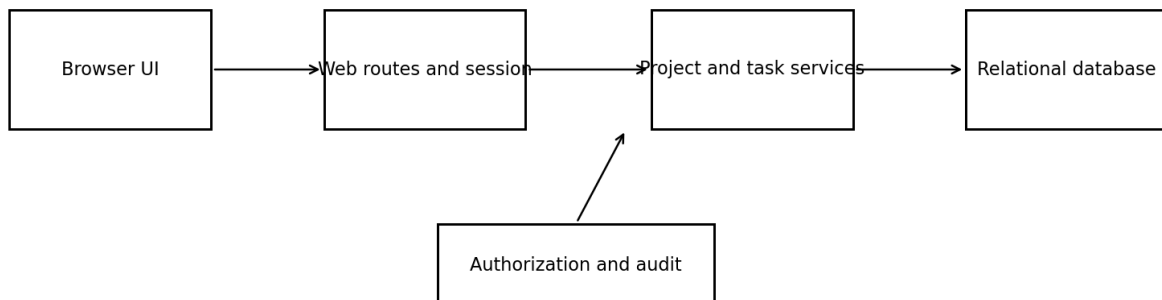


Figure 1. Web, service, authorization and persistence components.

3.4 Component descriptions

Component	Responsibility	SRS mapping
Browser UI	Projects, board, task detail, dashboard	FR-01–FR-17; NFR-04–06
Web routes / API	Session, request validation, JSON and CSV responses	FR-01–FR-17; SR-03
Authorization	Role and project membership checks before object access	FR-04, FR-06, SR-01
Project service	Creation, invitation, archive, notifications	FR-03, FR-04, FR-16, FR-17
Task service	Tasks, workflow, search, comments, time logs, summaries	FR-05–FR-15
Audit writer	Append actor, event and timestamp in same transaction	FR-07, FR-13
Relational database	Durable entities and indexed project-scoped reads	NFR-01–03

3.5 Architecture pattern and rationale

Use a layered monolith: presentation and routes, application services, persistence. One deployment and one database simplify transactions between task updates and audit entries. Services own business rules; views do not directly manipulate arbitrary project records.

3.6 Technology and data stores

Proposed stack: Django, server-rendered templates with JavaScript for drag-and-drop, PostgreSQL or SQLite for a classroom deployment. Data model: User; Project(id, owner_id, archived_at); Membership(project_id, user_id, role); Task(project_id, assignee_id, status, priority, estimate_hours, due_date); TimeEntry; Comment; ActivityEvent; Notification. Enforce unique membership per user/project and index task(project_id, status), event(project_id, created_at).

3.7 Risks and mitigations

Concurrent moves can overwrite one another: use row locking or an optimistic version field and return 409 for stale versions. A summary query can become slow: aggregate by project with suitable indexes and check NFR-01. An archive during an edit can race: recheck archived state in the same transaction.

3.8 Traceability to requirements

Project service: FR-03, FR-04, FR-16, FR-17. Task service: FR-05–FR-15. Account and session: FR-01–FR-02. Authorization: SR-01. Credential/session protection: SR-02–SR-03. Validation and logout: SR-04–SR-05. Database and UI: NFR-01–NFR-06. The test plan maps each requirement to a case.

3.9 Security architecture

Authenticate before every project route and apply a project membership policy at the service boundary. Check Owner/Manager/Member capability for each mutation and scope database queries by project ID. Django password hashing, CSRF middleware, secure cookie configuration for HTTPS deployment, output escaping and length validation protect credentials and content. Failed login counters expire after 15 minutes. Append audit records transactionally. Logs contain IDs and event types, never passwords or session secrets.

4 Design

4.1 Design overview

The API accepts a request, resolves the authenticated actor and project, checks role and archive state, validates fields, changes records in a transaction, appends an event where required and returns a documented status. Each task stores an integer version for concurrency checks.

4.2 UML sequence diagrams

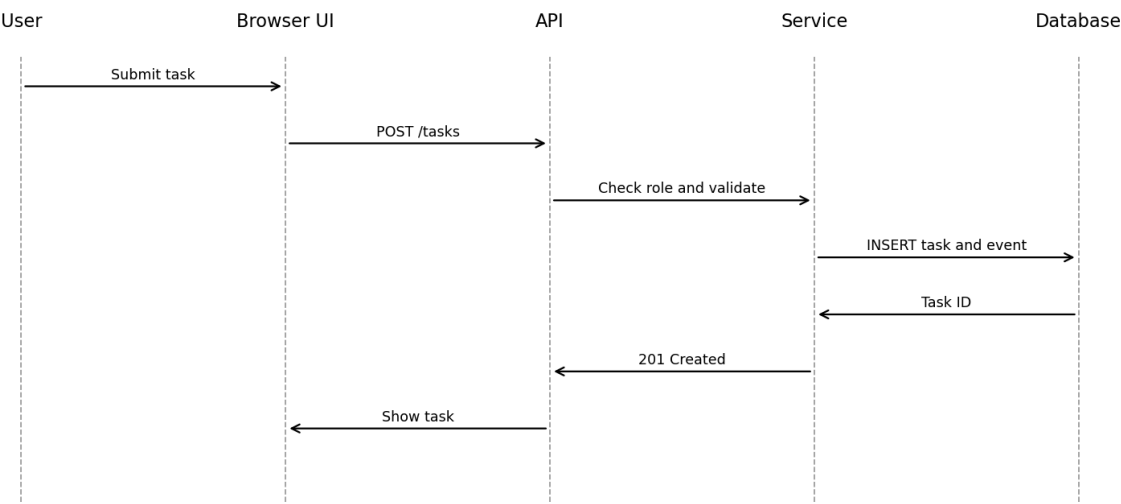


Figure 2. Task creation, permission check and audit transaction.

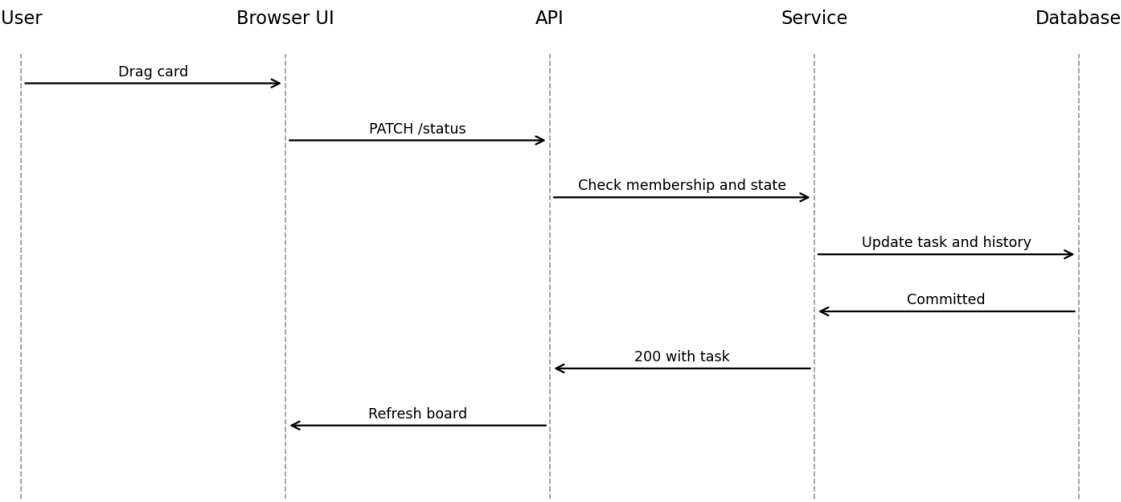


Figure 3. Kanban status update and conflict-aware response.

4.3 API design

Endpoint	Request	Success	Errors
POST /api/projects/{id}/tasks	JSON: title, description, assignee_id, priority, estimate_hours, due_date	201: id, status=Backlog, version=1	400 invalid; 403 role; 404 project
PATCH /api/projects/{id}/tasks/{task}/status	JSON: status, expected_versions	200: id, status, version	400 state; 403 member; 404 task; 409 stale/archive
POST /api/projects/{id}/tasks/{task}/time	JSON: hours, date, note	201: entry_id, actual_hours	400 amount; 403 member; 404 task
GET /api/projects/{id}/dashboard	No body	200: counts, overdue, estimated_hours, actual_hours	403 member; 404 project

GET /api/projects/{id}/tasks.csv	No body	200 text/csv	403 role; 404 project
----------------------------------	---------	--------------	-----------------------

All endpoints require a session; responses use JSON except CSV. Error body: {"error": {"code": "FORBIDDEN", "message": "Access denied"}}. IDs in paths are server-resolved and scoped to the project. State changes use CSRF protection.

4.4 Error handling, logging and monitoring

Use 400 for invalid input, 401 for missing session, 403 for denied action, 404 for inaccessible records and 409 for a stale task version or archived project. Roll back the update and audit together on failure. Log request ID, actor ID, project ID, endpoint, status and elapsed time; redact credentials and free-form content. Track error rate, failed sign-ins and slow board responses.

4.5 UX design

Show project switcher, board columns, searchable/filterable tasks, task detail and dashboard. Drag-and-drop has an equivalent keyboard status selector. Display estimate and actual hours separately. Confirm archive; keep archived project visibly read-only.

4.6 Open issues and next steps

Select PostgreSQL or SQLite based on deployment, choose invitation semantics for users without accounts, and record a fixed time zone display convention before implementation. Implement authorization first, then task workflow and tests. These choices do not alter SRS scope.

5 Appendices

Glossary: Kanban board = task columns by state; actual hours = sum of retained time entries; audit event = immutable record of a project action. Tools: Django test client, browser automation and load generator.